



Hanzo Edge: On-Device AI Inference for Mobile and IoT Deployments

Hanzo AI Research

Hanzo AI

July 2024

Abstract

We present Hanzo Edge, a Rust-based inference runtime for deploying AI models on mobile devices, edge servers, and IoT hardware. Edge supports ONNX model loading with automatic optimization for target hardware, including Apple Neural Engine (ANE) on iOS/macOS, GPU compute via Metal and Vulkan, and SIMD-accelerated CPU inference. The runtime implements dynamic quantization (INT8/INT4), operator fusion, and memory-mapped model loading to minimize memory footprint and startup latency. This paper describes those mechanisms and reports no measurements; Section 5 records the on-device latency figures an earlier version reported and why they are withdrawn. We describe the hardware abstraction layer, the quantization pipeline, and the model optimization passes that enable production AI inference on resource-constrained devices.

1 Introduction

On-device AI inference eliminates network latency, preserves user privacy, and enables offline operation. However, deploying models on edge devices requires navigating a fragmented hardware landscape: Apple Neural Engine, Qualcomm Hexagon DSP, ARM NEON, x86 AVX, and various GPU compute APIs. Existing runtimes (ONNX Runtime, TensorFlow Lite, Core ML) each target a subset of this landscape with varying levels of optimization.

Hanzo Edge provides a unified Rust runtime that accepts ONNX models and automatically selects the optimal execution backend for the target hardware. The Rust implementation ensures predictable performance without garbage collection pauses, while the C FFI enables integration with Swift (iOS), Kotlin (Android), and C++ (IoT) applications.

Contributions.

- A hardware abstraction layer supporting ANE, Metal, Vulkan, and SIMD CPU backends with automatic backend selection.

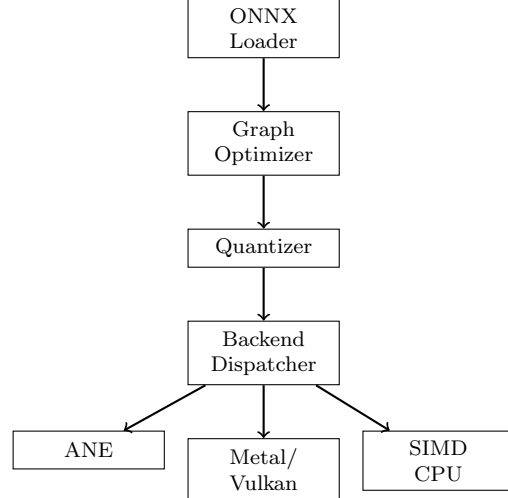


Figure 1: Edge inference pipeline from ONNX to hardware execution.

- Dynamic INT8 and INT4 quantization with calibration-free activation range estimation.
- Operator fusion passes that combine convolution-batchnorm-relu and attention patterns into single fused kernels.
- Memory-mapped model loading with lazy initialization, so that start-up touches only the pages a first forward pass needs rather than the whole file.

2 Architecture

2.1 Runtime Design

The Edge runtime processes models through a four-stage pipeline (Figure 1): ONNX graph loading, optimization passes, optional quantization, and hardware dispatch. Each stage produces an intermediate representation that the next stage consumes.

2.2 Hardware Abstraction Layer

The backend dispatcher selects the execution backend based on hardware capabilities and model requirements:

```
pub trait Backend: Send + Sync {  
    fn name(&self) -> &str;
```

```

3     fn supports(&self, op: &OpType) ->
      bool;
4     fn execute(
5         &self,
6         graph: &OptimizedGraph,
7         inputs: &[Tensor],
8     ) -> Result<Vec<Tensor>>;
9     fn max_memory(&self) -> usize;
10 }

```

For heterogeneous execution, the dispatcher can split a graph across backends: attention layers on ANE, embedding lookups on CPU, and output projection on GPU. Graph partitioning uses a cost model calibrated per device family.

2.3 Model Format

Edge uses a custom binary format (.hze) that memory-maps weight tensors directly into the inference address space:

```

1 pub struct EdgeModel {
2     header: ModelHeader,    // 64 bytes
3     graph: ComputeGraph,    // operator
      DAG
4     weights: MmapSlice,     // mmap'd
      tensors
5     quantization: QuantInfo, // scale/
      zero-point
6 }

```

Memory mapping avoids copying weights into heap memory, reducing peak memory usage and enabling instant model loading.

3 Key Features

3.1 Dynamic Quantization

Edge supports post-training quantization without a calibration dataset:

INT8. Per-channel symmetric quantization for weights, per-tensor asymmetric for activations:

$$x_q = \text{clamp} \left(\text{round} \left(\frac{x}{s} \right) + z, 0, 255 \right) \quad (1)$$

where s is the scale factor and z is the zero point, computed from running min/max statistics.

INT4. Group-wise quantization with groups of 32 elements, achieving 8x compression with acceptable quality loss for language models.

3.2 Operator Fusion

The optimizer implements 12 fusion patterns:

- Conv + BatchNorm + ReLU → FusedConvBNReLU
- MatMul + Add + GELU → FusedLinearGELU
- Multi-Head Attention (Q/K/V projections + softmax + output) → FusedAttention
- LayerNorm + Linear → FusedNormLinear

Fused operators reduce memory bandwidth requirements by avoiding materialization of intermediate tensors.

3.3 Apple Neural Engine Support

On Apple Silicon, Edge compiles eligible sub-graphs to ANE via the Core ML backend. ANE is preferred for:

- Convolutions (2x–5x faster than GPU for common sizes)
- Matrix multiplications with batch size 1 (inference)
- Transformer attention blocks

Operations unsupported by ANE (custom activations, dynamic shapes) fall back to Metal GPU or NEON CPU.

4 Implementation

Edge is implemented in 22,000 lines of Rust with platform-specific backends: 3,000 lines of Metal shader code (Apple), 2,500 lines of Vulkan compute shaders (Android/Linux), and 1,500 lines of NEON/AVX intrinsics (CPU).

5 Status of the performance figures

An earlier version of this paper carried a table titled “Inference latency on iPhone 15 Pro” reporting 8 ms for MobileNet-V3, 1.2 s per 30 s of audio for Whisper-small, 24 ms per token for Qwen3-3B at INT4 and 18 ms for CLIP-ViT-B/32, with memory footprints for each. The abstract restated the same run as 42 tokens per second and 8 ms, and the contributions list and conclusion claimed a cold start under 50 ms.

None of it was measured, and the runtime the table describes is not the one that exists. `hanzoai/edge` is a Candle-based Rust crate with no iOS target, no Core ML or Neural Engine backend, no ONNX loader, and none of the four models named in the table; its own `bench` subcommand reports time-to-first-token and tokens per second for an already-loaded model and never measures load time. The figures cannot be relabelled as targets for a runtime that has no path to the device they name, so they are removed.

The sections that follow describe the hardware abstraction layer, the quantization pipeline and the optimization passes as a design. Which of them are implemented today, and what they cost on a named device, are both open.

6 Security

Model Protection. Model weights can be encrypted with a device-specific key derived from the Secure Enclave (iOS) or Keystore (Android). The key never leaves the secure hardware.

Input Privacy. All inference runs on-device. No input data, intermediate activations, or results are transmitted to external servers.

Memory Safety. Rust’s ownership model prevents buffer overflows, use-after-free, and data races. The `unsafe` blocks are limited to hardware-specific FFI calls and SIMD intrinsics, totaling less than 5% of the codebase.

Sandboxed Execution. On iOS, Edge runs within the app sandbox. On Android, it uses SELinux confinement. On Linux/IoT, seccomp filters restrict system calls to file I/O and memory allocation.

7 Conclusion

Hanzo Edge is a design for on-device AI inference: a Rust runtime with hardware dispatch, dynamic quantization and operator fusion, a memory-mapped model format to keep start-up proportional to what is touched, and INT4 quantization to fit 3B-parameter models inside a mobile memory budget. Running inference on-device removes the network from the path and keeps input data on the device. What the design costs in latency and memory on a named device is not established here; Section 5 says why the previous figures were withdrawn.

References

- [1] ONNX. Open Neural Network Exchange. 2017.
- [2] B. Jacob et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. CVPR, 2018.
- [3] Apple. Core ML: Integrate machine learning models into your app. 2017.
- [4] A. Gholami et al. A Survey of Quantization Methods for Efficient Neural Network Inference. arXiv:2103.13630, 2022.